CSE2115 Peer Assignment 6 - DevOps

REDACTED

Technische Universiteit Delft

1. Observability

1.1. Scenario

- (1) **Senario** An attacker attempts to use a known list of stolen cards (and their details) on the payment page. They send thousands of payments from only a handful of IP addresses. This extra load slows the payment gateway and real customers experience waits or errors when they try to checkout.
- (2) **What goes wrong:**
  - When the fraud score is above 0.8, processPayment() returns "declined" before any data is stored. As a result blocked attempts leave no trace. We cannot prevent them more easily in the future.
  - Nobody counts payments per IP or per user. The attack then looks like traffic. By saving information (without breaking legal limits for real customers) we can spot weird requests and malicious traffic.
  - Nobody measures how long the gateway takes. The slowdown is only seen when customers complain.
  - Then no one can know whether the cause is the code, the fraud service, the gateway or the database because no time trace is kept..RE

  It is hard to detect because every single request looks the same. The problem appears in the pattern of **many attempts & slow responses**. We collect nothing that shows that pattern or any pattern all.

1.2. Observability points

We never log full card numbers (due to privacy reasons). We only log items such as user id, payment method type and a request id.

- 1. Start of processPayment()
  - What: request id, user id, amount, currency, payment method type, IP address, time.
  - How: a structured log line (format, easy to search) a counter metric for payment requests and the start of a trace (to check time).
  - Goal: both. Security: we see who pays, from where and how often. Many requests from one IP is a warning sign. Performance: the counter shows traffic and the trace measures the payment..RE
- 2. Fraud detection call (analyzeTransaction)
  - What: response time, errors or timeouts and the fraud score.
  - How: a timing a step in the trace and a log of the score.
  - Goal: both. Performance: it is a service so we must see when it is slow or down. Security: scores per user or IP show who’s suspicious and a sudden change in normal scores can mean an attack or a broken fraud service..RE
- 3. Fraud decline (score > 0.8)
  - What: user id, IP, fraud score, reason.
  - How: a warning log and a counter "declined for fraud" with an alert if it jumps.

    - Goal: security. This is where blocked attempts disappear today. The counter shows an attack in progress. The logs let us investigate afterwards.
- 4. Gateway charge (gateway.charge)
    - What: response time, status or error code, timeouts.
    - How: a timing metric a failure counter, a step in the trace and a log when it fails.
    - Goal: both. Performance: it is the important external call so it shows if the gateway causes slow check-outs. Security: many failed charges in a row are typical, for card testing.

2. Key performance indicators

2.1. Project Forum

2.1.1. Health KPI's

- Response time
    - Metric: the time it takes to handle 95% of requests, and the moving average of the slowest 5 responses in the last hour
    - Why: This metric allows you to see if a change affected the average response time, not counting outlier, and it shows if the outliers have worsend.
    - Problem: average response times above 1 second should be unacceptable. Any differences should be monitored just after a release. Outliers are harder to quantify and should mostly be compared before and after deploying an update.
- Resource utilization
    - Metric: The percentage of memory and CPU being used
    - Why: A small amount of servers means a small amount of resources, identifying inefficient changes to be able to keep overhead low is a high priority
    - Problem: any suden changes might indicate a server error. And prolonged high usage, say 80% for 10 minutes or memory that never gets freed could indicated bad changes.

2.1.2. Quality KPI's

- Bug reports per release, per visitor
    - Metric: The amount of bugreports within a set time frame after release, for example a week, divided by the number of users the platform saw in that week
    - Why: It shows how many visitors are encountering certain issues and how stable the platform is.
    - Problem: If after an update the metric sudenly spikes we know we have introduced more bugs than we have solved or we have degradated the platforms stability
- Average time spent on the website
    - metric: the average time a user spents using the website per session
    - Why: the longer a user has to spend to complete a task the worse the UI functions, it's either unintuative, or unresponsive. Both could indicate a recession in functionallity.
    - Problem: If sudenly users are taking longer to complete tasks within the site the site has become more cumbersome to work with after the update, and this should be fixed

2.2. Spotify

2.2.1. Health KPI's

- Playback failure rate
  - Metric: When a user presses play but a song does not start playing, not couting offline users.
  - Why: The app is used by users that want to listen to music, it's the primary core functionality that users expect. failure's should be extremely low
  - Problem: failure should be extremely low, since a 1% difference across milions of listeners means ten's of thousands of users who are experiencing issues. We think atleast 99.6% and a difference not larger than 0.1 per update.
- Time to music start
  - Metric: the average time it takes the app from the user opening the app to the first music being played
  - Why: Spotify is used for music, when a user decides they want music they want it NOW. So the app should load quickly and music should not take long to fetch
  - Problem: Any sudden drop in average or the amount of outliers should be seen as a incident to be corrected

2.2.2. Quality KPI's

- Crash-free sessions
  - Metric: the percentages of sessions that doesn't end on a crash
  - Why: Spotify serves a lot of individuals and none of them like being interrupted, Having to restart a session because it crashed is a horible user experience
  - Problem: any suden drop should again not be tolerated, but we should aim for atleast 99% or higher
- Listening time per user
  - Metric: the average time users spend listening
  - Why: on average users should listen a very consistent amount, if the app is working and the music isn't being interrupted or buffering. hence this could indicate poor content delivery. It also tied in directly with gross profits and therefore the buisness is probably interested in this metric.
  - Problem: Again, any suden drop indicates a bad update and should be looked at. Aditionally the amount of outliner should be analized to see if any rare but critical bugs have been introduced.